INTEGRATION GUIDE

메신저 링크 · 첨부파일 클릭 연동 가이드

PDF 등 첨부파일을 눌렀을 때 닫기 버튼 없는 화면이 열리는 문제를 외부 뷰어 연동으로 해결하는 방법입니다.

대상 · 모바일 웹뷰 담당 개발자 (iOS / Android)

SDK · @sendbird/ai-agent-messenger-react

예제 코드 · `web/src/libs/openExternalLink.ts`

1 현상과 원인

> 채팅에 첨부된 **PDF를 누르면 닫기 버튼이 없는 화면**이 열려 이전 화면으로 돌아갈 수 없습니다. 이미지를 누를 때는 정상적으로 닫기 버튼이 보입니다.

이미지와 PDF는 처리 경로가 다릅니다

SDK는 **이미지·동영상만 내부 뷰어**로 표시합니다. 그 외 파일(PDF 포함)과 마크다운 링크·인용·CTA 버튼은 **새 창 열기를 요청**합니다.

```
이미지 · 동영상    →  SDK 내부 뷰어          →  닫기 버튼 있음 ✅

PDF · 기타 파일    →  window.open(url)     →  앱이 처리        →  ?
마크다운 링크      →  window.open(url)     →  앱이 처리        →  ?
인용 · CTA 버튼    →  window.open(url)     →  앱이 처리        →  ?
```

`window.open`은 **요청일 뿐**이고, 실제로 어디에 어떻게 열지는 전적으로 앱이 정합니다. 웹뷰 안에서는 다음 문제가 생깁니다.

환경	기본 동작과 문제
iOS (WKWebView)	앱이 `WKUIDelegate`로 처리하지 않으면 **무시**됩니다. 처리하더라도 닫기 UI 없는 새 `WKWebView`를 띄우면 **돌아갈 방법이 없습니다.**
Android (WebView)	같은 WebView에서 열면 **챗봇 화면을 덮어버립니다.** 뒤로가기 처리가 없으면 대화로 복귀할 수 없습니다.

2 해결 방식

SDK가 제공하는 `handlers.onClickLink`로 **`window.open`이 호출되기 전에 가로채서** 브릿지로 네이티브에 넘깁니다. 네이티브는 **OS 기본 외부 뷰어**로 열면 되고, 이 뷰어들은 **닫기/완료 버튼이 기본 제공**됩니다.

```
[변경 전]  PDF 클릭  →  window.open  →  닫기 UI 없는 새 WebView  ❌

[변경 후]  PDF 클릭  →  onClickLink  →  브릿지 open_external  →  OS 외부 뷰어 ✅
                                                              (닫기 버튼 기본 제공)
```

"챗봇이 호출한 새 창인지" 구분값이 필요 없습니다

네이티브에서 `window.open` 을 가로채 판별하려 하면 페이지의 다른 스크립트가 띄운 새 창과 구분할 수 없어 별도 식별값 설계가 필요합니다.
이 방식은 **전용 브릿지 이벤트(`open_external`)로만 도착**하므로 출처가 챗봇임이 구조적으로 보장됩니다.
`WKUIDelegate` 나 `shouldOverrideUrlLoading` 을 건드릴 필요도 없습니다.

3 웹 구현

링크 핸들러

```
// web/src/libs/openExternalLink.ts
import { bridge } from './bridge';

export function openExternalLink({ url }: { url: string }): void {
  if (!url) return;

  // SDK 기본 동작과 맞춥니다 — 프로토콜이 없으면 https 를 붙입니다.
  const normalized = /^https?:\/\//i.test(url) ? url : `https://${url}`;

  if (bridge.isNative()) {
    // 네이티브 앱 안에서 실행 중 → 외부 뷰어로 위임
    bridge.send('open_external', { url: normalized });
    return;
  }

  // 일반 브라우저에서는 SDK 기본 동작을 그대로 유지
  window.open(normalized, '_blank', 'noopener,noreferrer');
}
```

메신저에 연결

```
import { openExternalLink } from '../libs/openExternalLink';

<FixedMessenger
  appId={...}
  aiAgentId={...}
  // 메신저 안의 링크·첨부파일 클릭을 네이티브 외부 뷰어로 연결
  handlers={{ onClickLink: openExternalLink }}
>
```

적용 범위

`onClickLink` 는 **마크다운 링크, 관리자 메시지 URL, 인용 링크, CTA 버튼, 이미지가 아닌 첨부파일 미리보기 클릭**에 모두 적용됩니다. 이미지·동영상의 내부 뷰어와 뷰어 안의 다운로드 버튼은 영향을 받지 않습니다. (다운로드는 브라우저 기본 동작이 유지되도록 의도적으로 제외되어 있습니다)

4 iOS 구현

`SFSafariViewController`를 사용합니다. **완료(닫기) 버튼과 공유 기능이 기본 제공**되고 PDF 렌더링도 OS가 담당하므로, 웹뷰에 직접 띄울 때 생기는 문제가 발생하지 않습니다.

```
import SafariServices

// 브릿지 이벤트 처리
case "open_external":
    guard let payload = data as? [String: Any],
          let urlString = payload["url"] as? String,
          let url = URL(string: urlString),
          let scheme = url.scheme?.lowercased(),
          scheme == "http" || scheme == "https" else {
        print("[Bridge] open_external: 유효하지 않은 URL")
        return
    }
    DispatchQueue.main.async { self.presentExternalViewer(url) }


private func presentExternalViewer(_ url: URL) {
    guard let scene = UIApplication.shared.connectedScenes
            .compactMap({ $0 as? UIWindowScene })
            .first(where: { $0.activationState == .foregroundActive }),
          let root = scene.windows.first(where: { $0.isKeyWindow })?.rootViewController else {
        return
    }
    // 이미 모달이 떠 있으면 그 위에 표시합니다.
    var top = root
    while let presented = top.presentedViewController { top = presented }

    let viewer = SFSafariViewController(url: url)
    viewer.dismissButtonStyle = .close
    top.present(viewer, animated: true)
}
```

앱 밖으로 내보내려면

Safari 앱으로 전환하고 싶다면 `UIApplication.shared.open(url)` 을 쓰면 됩니다. 다만 앱을 벗어나므로, 대화로 자연스럽게 복귀하려면 `SFSafariViewController` 쪽이 유리합니다.

5 Android 구현

`ACTION_VIEW`로 넘기면 OS가 기본 브라우저·PDF 뷰어를 띄우므로 **뒤로가기와 닫기가 기본 제공**됩니다.

```
// 브릿지 이벤트 처리
"open_external" -> {
    val url = data?.optString("url").orEmpty()
    mainHandler.post { activity.openExternalViewer(url) }
}
```

```
private fun openExternalViewer(url: String) {
    val uri = runCatching { Uri.parse(url) }.getOrNull()
    val scheme = uri?.scheme?.lowercase()
    if (uri == null || (scheme != "http" && scheme != "https")) {
        Log.w("Bridge", "open_external: 유효하지 않은 URL — $url")
        return
    }
    val intent = Intent(Intent.ACTION_VIEW, uri).apply {
        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    }
    try {
        startActivity(intent)
    } catch (e: ActivityNotFoundException) {
        // 처리 가능한 앱이 없는 경우 — 사용자에게 알리고 조용히 넘어갑니다.
        Toast.makeText(this, "파일을 열 수 있는 앱이 없습니다.", Toast.LENGTH_SHORT).show()
    }
}
```

URL 스킴 검증은 반드시 넣으세요

`http` / `https` 만 허용합니다. 검증 없이 `ACTION_VIEW` 나 `SFSafariViewController` 에 넘기면 의도하지 않은 스킴(`file://` , 커스텀 스킴 등)으로 앱 내부 화면이 열릴 수 있습니다.

6 동작 확인

확인 항목	기대 결과
PDF 첨부 클릭	외부 뷰어가 뜨고 **완료/뒤로 버튼**으로 대화에 복귀
이미지 첨부 클릭	기존과 동일하게 SDK 내부 뷰어 (변화 없음)
답변 속 링크 클릭	외부 뷰어로 열림
일반 브라우저에서 열기	`window.open` 기본 동작 유지 (새 탭)
대화 복귀 후 상태	대화 내용과 스크롤 위치가 유지됨

참고

웹 쪽 수정만으로는 동작하지 않습니다. **웹의 `onClickLink` 연결과 네이티브의 `open_external` 처리가 둘 다 있어야** 완성됩니다. 한쪽만 적용하면 링크를 눌러도 아무 일이 일어나지 않습니다.

참고 문서

SDK 공식 가이드 — How to handle link clicks in conversation messages

`sendbird/ai-agent-js` → `.discussions/conversation/handle-link-taps/react.md`

네이티브 SDK를 직접 쓰는 경우 같은 디렉터리의 `ios.md` / `android.md`에 해당 플랫폼 구현이 정리되어 있습니다.